

MiniMap Modular (Radar)

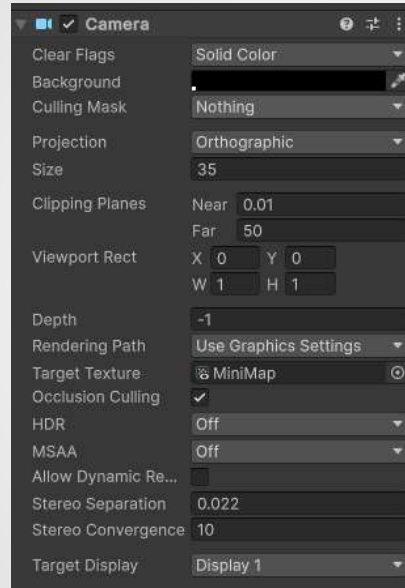
Add mini maps and radar to your games easily! Some things can be customized by the inspector and you can see them being changed.

The package includes a bunch of images for you to customize as you see fit. If you want to make your own, be sure to study the images and, if possible, overwrite the existing ones to follow the proportions.

Feel free to use them in your most varied projects and I am very grateful for having purchased our package. If you have any questions or need help, please contact us and I will help you as soon as possible.

W.L.O Games Developing Dreams.

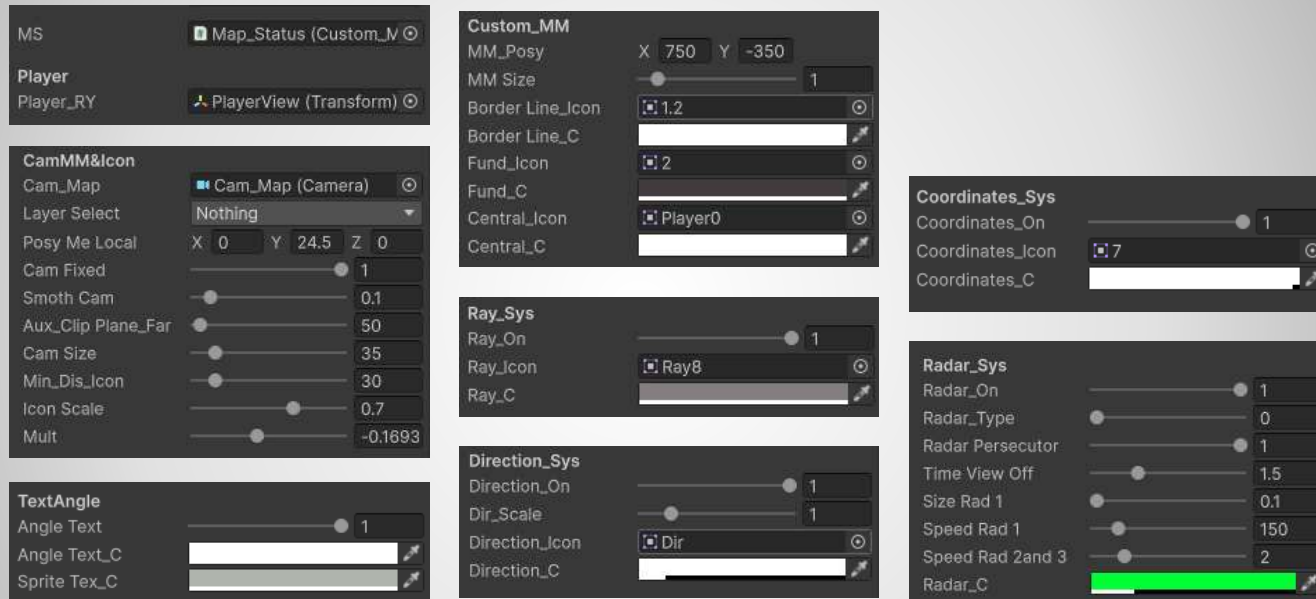
Câmera



Here is how Cam_Map was configured. It is possible to use its view to show the map, but I believe that depending on the project, this may display an error when entering interiors unless there is a script or good control of what the camera can see and place it at an acceptable height.

Some of its settings are made and applied by the responsible script that we will see later.

Note: there are two cameras in the project, configure the character's camera as you wish, only the mini map camera requires some care.



MS: I will talk more about it later, but basically it is a script that serves as an auxiliary for variables that need to be accessed by the MP. I could let it be linked automatically, but to avoid errors, since it is used a lot and since it is something that will start with the player, it can be easily linked.

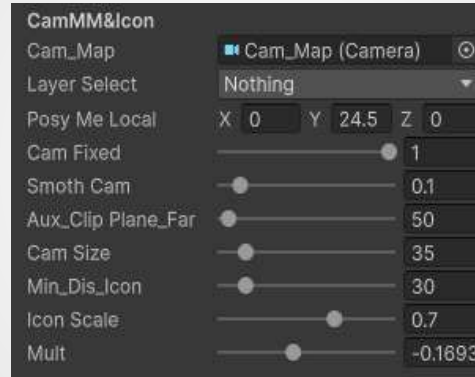
Player_RY: transform where the player rotates on the y axis to feed systems that need this information, such as coordinates and icons.

_C: where it has, it means that it changes the color of the corresponding item. Please check if the opacity is different from 0 so that it does not become invisible.

_Icon: where it has, it means that it changes the sprite of the corresponding item. Do not leave it null.

_On: where it has, it means that it is possible to activate this item by setting the value to 1;

MP - CamMM&Icon



Cam_Map: The camera that will be responsible for the mini map.

LayerSelect: What the camera can see, use this to see part of the scene or a plane of the scene.

PosyMeLocal: What position the camera will be in relation to the player, rotation locks x= 90.

CamFixed: Indicates whether the camera is the player's child or if it only follows him, used to rotate everything and not just the player icon.

SmothCam: Smoothing of camera movement.

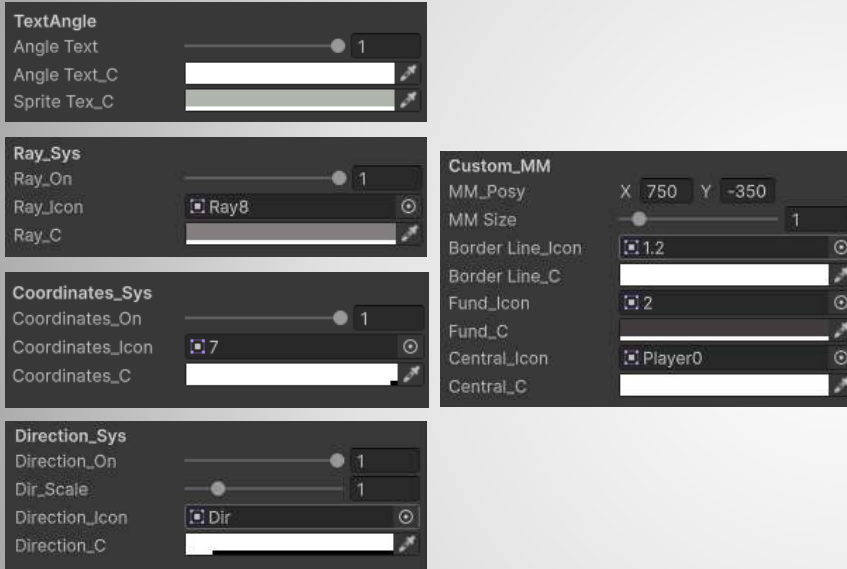
AuxClipPlaneFar: Changes the camera's (Far) value.

CamSize: Changes the camera's (Size) value.

MinDisIcon: Generally should be less than 5 from the CamSize value, used when the icon is more than 30 away to show the directional target.

IconScale: Size of the icon on the mini map.

Mult: when adjusting parameters of this part it may be necessary to adjust the icons, activate the camera view and check if the icons are against their targets if not just adjust here until aligned and save this value and then enter it check that it is correct and you can disable the camera view.



Ray_Sys: Ray sprite for decoration.

Coordinates_Sys: Coordinates sprite that rotates depending on the player's rotation to give an idea of cardinal directions.

Direction_Sys: Get an idea of the direction the player is looking, being able to change its size on the x-axis.

AngleText: Activates a text section indicating the angle of the player's rotation.

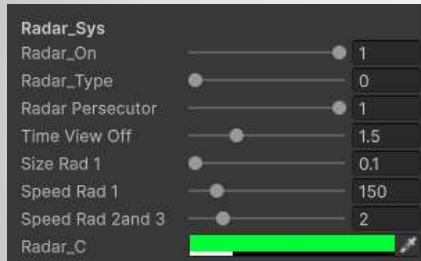
_C: where it has it means that it changes the color of the corresponding item, please check if the opacity is different from 0 so that it does not become invisible.

_Icon: where it has it means that it changes the sprite of the corresponding item, do not leave null.

_On: where it has it means that it is possible to activate this item leaving the value equal to 1;

MMPosy: position where you want the mini map.

MMSize: size you want the mini map.



Radar_Sys: Used to see hidden icons.

_On: where it is, it means that it is possible to activate this item by leaving the value equal to 1;

RadrType: choose the desired radar type, do not change it during play.

RadarPersecutor: with its value equal to 1, the hidden icons continue to move while they are under the effect of the radar.

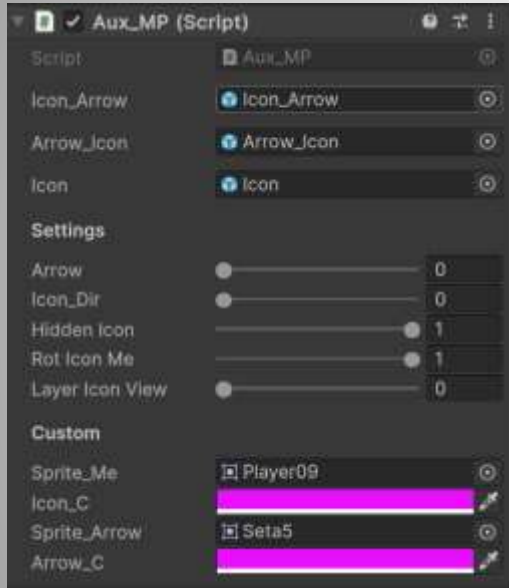
TimeViewOff: you want the hidden icon to remain visible until the radar effect passes.

SizeRad1: size of radar 1 on the x axis.

SpeedRad1: rotation speed of radar 1.

SpeedRad2and3: growth speed of radars 2 and 3.

_C: where it is, it means that it changes the color of the corresponding item, please check if the opacity is different from 0 so that it does not become invisible.



Arrow: Enables the arrow on the mini map.

Icon_Dir: Enables the inventory icon.

Hidden_Icon: Makes the icon hideable and visible only with the radar.

Rot_Icon_Me: Makes the icon rotate according to the reference item.

Layer_Icon_View: View priority (0,1,2), in case you want to separate it into layers.

Note: Only one can be enabled at a time. If you try to enable both, Arrow will be enabled. HiddenIcon does not allow you to enable Arrow or Icon_Dir.

Icon_Arrow, Arrow_Icon, Icon: These are the pre-made items used in the mini map. There is no need to change them as we usually change the image and color here. If you want to change the position of the Arrow_Icon icon, leave it at $y = 140$.

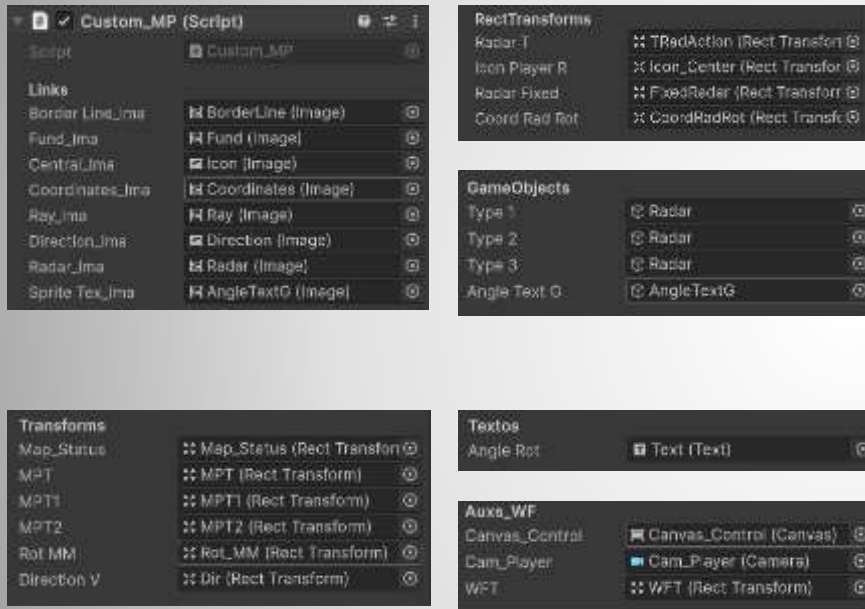
Sprite_Me: The sprite you want for the item.

Icon_C: Color of the item, check if the opacity is at 255.

Sprite_Arrow: The arrow sprite. If you activate the arrow, don't forget to link it.

Arrow_C: Color of the arrow, check if the opacity is at 255.

Custom_MP



Links:

Transforms:

RectTransforms:

GameObjects:

Texts:

Auxs_WF:

Note: information like this is basically links that are accessed to help the package become more practical by providing locations and data related to the necessary variables. Be careful when changing them as this may cause problems with its operation.

I like to be detailed, so I'll give some examples.

Basically, links provide images so they can be edited by changing the sprite and color and providing location.

Transforms and RectTransforms are used to access scale, rotation and position, so much so that RotMM is used to rotate coordinates, etc.

GameObjects are used to facilitate the manipulation of certain objects, such as activating and deactivating the type of radar chosen.

Texts are used to link the texts used, there's only the one that shows the angle.

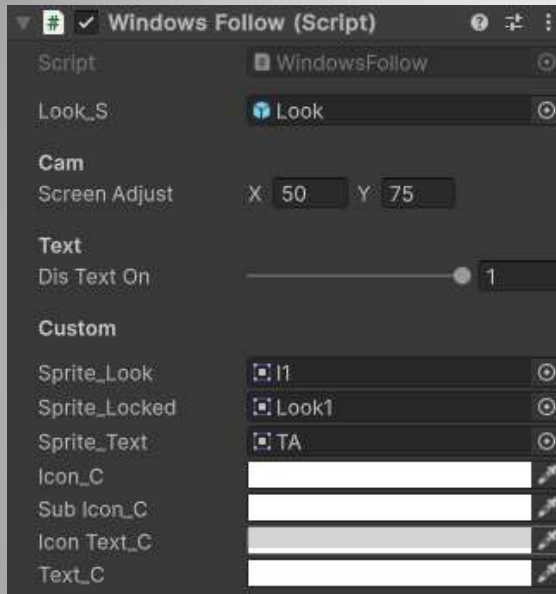
Auxs_WF I need to talk about this one, we'll talk more about how to use this data later.

Canvas_Control: Canvas used, of course, there's only one in the project.

Cam_Player: Camera used in the player's view.

WFT: Location of where the WindowsFollow children will be born.

WindowsFollow



Let me make a note: this script is responsible for leaving a directional icon for an objective on the screen. I'll leave a picture for better visualization.

Look_S: pre-made icon.

ScreenAdjust: these are the limits of the icon, keeping it always within the screen or even increasing the screen margin.

DisTexOn: as seen previously, it activates the text showing its distance from the player.

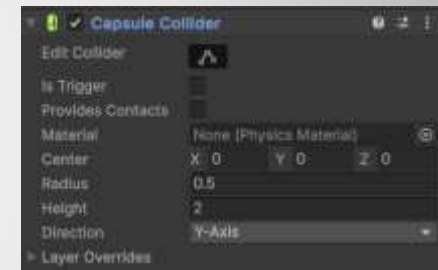
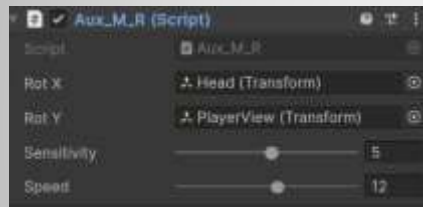
Custom: there are sprites that can be changed and colors that can be changed.

Look: this is the question mark image in this case, the base icon.

Locked: these are the spaces around it that show that the icon is on the screen, taking the measurements of ScreenAdjust.

Note: if you want it to appear over the mini map, place (WFT) in front of (Map_Status). I recommend changing the opacity so that it becomes transparent and does not interfere with the mini map and its UI.

Aux_M_R



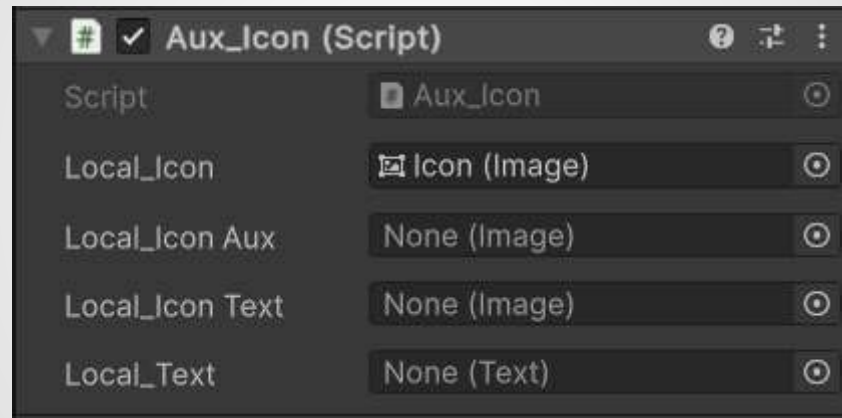
Simple player control and camera rotation script, there's not much to say.

Set the transform that you want to rotate each axis (XY).

Set the desired movement and rotation speed within the allowed range.

I'll leave the configuration just so it doesn't look too empty here.

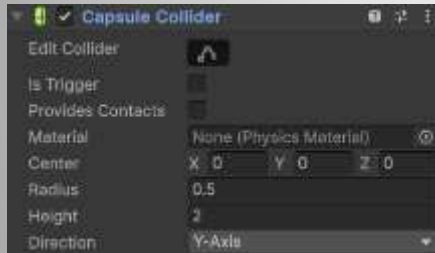
Aux_Icon



Basically, it helps in finding items to be used, not all objects that will use all functions, only WindowsFollow that will use all of them.

They are basically links to the subitems of that object.

Simple_AI



RandomAction: If the value is 1, it will follow the waypoints randomly, if disabled it will follow the order of the Targets.

DisNTarget: Minimum distance that the enemy moves to the next target when it reaches the target.

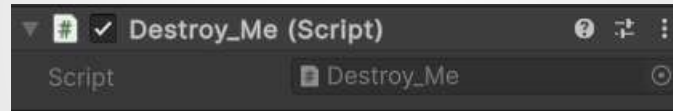
SpeedMe&SpeedRot: Movement and turning speed.

Targets: Waypoints to be used in the script patrol

What I can say about this script is that I basically created something just to make it easier to demonstrate the script so that it doesn't just have stationary objects. I'll leave a photo of the Ai_ configuration.

Note: it's not a very elaborate script because that's not the purpose of this package, but I ended up almost creating something more professional, but that's for another package.

Destroy_Me



I did more to test the destruction system, when pressing the letter "P" the object that has this script is deleted, every object that generates an icon when destroyed it destroys its icon.